

SIMATS ENGINEERING
ASSESSMENT TOOL 2 – APPLICATION-BASED QUESTIONS (High)

Course:	Code:	CO: CO5
Duration:	Total Marks: 100	Reg No / Name :

COURSE OUTCOME COVERED

CO5: *Analyze vulnerabilities in Windows and Linux systems and evaluate security weaknesses in messaging, web, and mobile applications using appropriate exploitation techniques and hacking tools. (BT5)*

Instructions: Answer all questions. Each question carries 20 marks.

1. A mid-sized financial organization operates several legacy Windows servers where multiple RPC services remain exposed due to delayed patching cycles caused by operational constraints. Recently, abnormal traffic patterns and unauthorized connection attempts have been observed within the internal network. As a security analyst, evaluate how attackers could exploit these RPC vulnerabilities to gain unauthorized access and escalate privileges. Assess the level of risk associated with delaying patches in such environments and justify whether immediate patch deployment or controlled maintenance scheduling is more appropriate. Recommend suitable mitigation strategies to reduce exposure without affecting business continuity.
2. A web application used for online transactions suffers from multiple vulnerabilities, including cross-site scripting (XSS), cross-site request forgery (CSRF), and weak session management practices. Users report unauthorized transactions and session hijacking incidents. Evaluate how attackers could combine these vulnerabilities to execute complex attacks affecting user accounts. Assess the risks associated with poor session handling and lack of input validation. Justify the need for a layered security approach and recommend comprehensive mitigation strategies to secure the application.
3. An enterprise uses an outdated IIS server to host multiple internal and external applications, and recent vulnerability scans indicate several unpatched critical flaws. Despite warnings, the organization delays updates due to fear of downtime and service disruption. Assess how attackers could exploit these vulnerabilities to gain unauthorized access or execute remote code. Evaluate the risks associated with delaying security updates in such environments and justify whether immediate patching or temporary

isolation is more appropriate. Recommend strategies to ensure both security and service availability.

4. A mobile application used by customers stores sensitive information such as login credentials and personal data in plaintext within the device storage. Additionally, the application communicates with backend servers over unsecured channels. Evaluate how attackers could exploit these weaknesses through device compromise or network interception. Assess the potential impact on user privacy and organizational reputation. Justify the need for encryption and recommend secure storage and communication techniques.
5. A system administrator disables the Windows firewall temporarily to troubleshoot connectivity issues but forgets to re-enable it. Within a short period, multiple unauthorized access attempts are detected. Evaluate how attackers could exploit this temporary exposure to gain access to critical services. Assess the risks associated with mismanagement of security controls and justify the importance of enforcing automated policies. Recommend administrative safeguards to prevent such incidents.

Code of Conduct:

I ____M.NITHIYASHREE-192421101____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:M.NITHIYASHREE

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Mark
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem	
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts	

Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach	
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution	
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand	

1. Legacy Windows RPC Vulnerabilities

Problem Analysis

The organization operates legacy Windows servers with multiple exposed RPC services. Due to delayed patching, attackers may identify vulnerable RPC services and attempt to exploit them to gain unauthorized access. Once initial access is obtained, attackers may attempt privilege escalation and use the compromised server to access other internal resources.

How Attackers Could Exploit RPC Vulnerabilities

RPC allows Windows systems and applications to communicate across a network. When vulnerable RPC services are exposed, attackers may:

1. **Identify exposed RPC services** through network and service discovery.
2. Determine whether the services are outdated or affected by known vulnerabilities.
3. Attempt to exploit vulnerable services to obtain unauthorized access.
4. If successful, attackers may execute unauthorized operations with the privileges available to the exploited service.
5. Attackers may then attempt **privilege escalation** to obtain higher-level administrative access.
6. A compromised server can potentially become a starting point for further internal attacks.

The risk becomes greater when the vulnerable systems contain sensitive financial information or have connections to other critical systems.

Risk of Delaying Patches

Delaying security updates creates a **high security risk** because:

- Known vulnerabilities remain exploitable.
- Attackers may already have publicly available information about the vulnerabilities.
- The longer systems remain unpatched, the greater the exposure window.
- A compromised legacy server may be used for lateral movement.
- Financial organizations are attractive targets for attackers.
- Recovery from a successful attack may cause greater downtime than planned maintenance.

Immediate Patching vs Controlled Maintenance

Immediate patching should be prioritized for critical vulnerabilities, especially when active exploitation or suspicious traffic has already been observed.

However, business-critical systems should not be patched blindly. A **controlled emergency maintenance process** is preferable:

Test the patch → back up systems → deploy to a small group → monitor → expand deployment.

For systems where immediate patching is technically impossible, temporary isolation should be used until the patch can safely be deployed.

Mitigation Strategies

- Apply security patches according to risk priority.
- Disable unnecessary RPC services.
- Restrict RPC access using firewalls and network segmentation.
- Allow RPC only from trusted systems where possible.
- Monitor unusual RPC traffic.
- Use endpoint detection and response (EDR).
- Maintain regular vulnerability scanning.
- Keep reliable backups.
- Test patches before large-scale deployment.
- Establish an emergency patch-management procedure.

Conclusion

Delaying patches on exposed legacy Windows servers creates a high risk of unauthorized access, privilege escalation, and lateral movement. Because abnormal traffic and unauthorized connections have already been observed, critical patches should receive immediate attention through a controlled maintenance process. Where immediate patching is impossible, affected systems should be isolated and access restricted. This approach provides security while minimizing business disruption.

2. Web Application – XSS, CSRF and Weak Session Management

Problem Analysis

The web application handles online transactions but contains **XSS, CSRF, and weak session-management vulnerabilities**. These weaknesses can become more dangerous when combined because an attacker may use one vulnerability to support another attack.

How the Vulnerabilities Can Be Combined

1. Cross-Site Scripting (XSS)

XSS occurs when an application improperly handles user-controlled input and allows malicious script content to execute in another user's browser.

A successful XSS attack may allow an attacker to perform unauthorized actions in the victim's browser or access information that should not be exposed to untrusted scripts.

2. Cross-Site Request Forgery (CSRF)

CSRF tricks an authenticated user's browser into sending an unintended request to a trusted application.

For example, if transaction requests are not properly protected, an attacker may attempt to cause a logged-in user to perform an unauthorized action.

3. Weak Session Management

Poor session management can include:

- Long-lived sessions
- Predictable session identifiers
- Failure to invalidate sessions after logout
- Missing secure cookie settings
- Improper session timeout

These weaknesses increase the possibility of **session hijacking and account takeover**.

Combined Attack Risk

An attacker could potentially combine these weaknesses as follows:

XSS vulnerability → malicious activity in victim's browser → abuse of authenticated session → unauthorized transaction

Weak session controls can make the impact worse because stolen or improperly protected session information may remain useful for longer.

Risks

The organization could face:

- Unauthorized transactions
- Account takeover
- Theft of personal information
- Financial losses
- Privacy violations
- Loss of customer trust
- Regulatory consequences
- Reputational damage

Layered Security Approach

A single security control is not sufficient.

The application should use **defense in depth**, including:

Input Layer → Application Security → Session Security → Request Validation → Database Security → Monitoring

Mitigation Strategies

Against XSS

- Validate input.
- Encode output appropriately.
- Use Content Security Policy (CSP).
- Avoid unsafe handling of user-controlled HTML.
- Apply secure development practices.

Against CSRF

- Use anti-CSRF tokens.
- Use appropriate **SameSite** cookie settings.
- Validate request origin where appropriate.
- Require additional verification for sensitive transactions.

Against Session Attacks

- Generate strong, unpredictable session identifiers.
- Use HTTPS.
- Set cookies with **Secure** and **HttpOnly**.
- Apply appropriate session expiration.
- Invalidate sessions after logout.
- Regenerate sessions after authentication.

Conclusion

XSS, CSRF, and weak session management can interact to create serious account-security and transaction risks. The organization should adopt a layered security approach rather than depending on a single control. Input validation, output encoding, CSRF protection, secure cookies, HTTPS, strong session management, transaction verification, logging, and continuous security testing should be implemented to protect customer accounts.

3. Outdated IIS Server – Unpatched Critical Vulnerabilities

Problem Analysis

The organization uses an outdated IIS server for internal and external applications. Vulnerability scanning has identified critical unpatched vulnerabilities, but updates are being delayed because of concerns about downtime.

This creates a significant security exposure.

How Attackers Could Exploit the Vulnerabilities

At a high level, an attacker could:

1. Identify the externally accessible IIS server.
2. Determine its version and exposed services.
3. Identify known vulnerabilities affecting that version.
4. Attempt to exploit a vulnerable component.
5. If successful, obtain unauthorized access or cause unauthorized code execution.
6. Attempt privilege escalation or access application resources.
7. Use the compromised server as a pathway toward other internal systems.

The risk is particularly high for an internet-facing server because attackers do not necessarily need internal network access to begin their attacks.

Risks of Delaying Security Updates

Delayed updates can result in:

- Exploitation of publicly known vulnerabilities.
- Unauthorized access.

- Remote code execution.
- Data theft.
- Malware deployment.
- Website/application disruption.
- Lateral movement.
- Financial and reputational losses.

The concern about downtime is understandable, but **leaving critical vulnerabilities unpatched can create much greater operational disruption** if an attack succeeds.

Immediate Patching vs Temporary Isolation

For critical vulnerabilities, **immediate risk reduction should be prioritized**.

The preferred approach is:

Backup → Test patch → Controlled deployment → Monitoring

If patching cannot immediately be performed, the vulnerable IIS server should be **temporarily isolated or protected** using network controls until patching is completed.

Temporary isolation should be considered a **short-term mitigation**, not a permanent replacement for patching.

Strategies for Security and Availability

- Maintain tested backups.
- Use staging/test environments before production updates.
- Schedule maintenance during low-traffic periods.
- Use redundant servers where possible.
- Apply patches in phases.
- Use firewall rules to restrict unnecessary access.
- Use reverse proxies/WAF where appropriate.
- Disable unnecessary IIS components.
- Continuously monitor server logs.
- Perform regular vulnerability scanning.
- Maintain a documented patch-management policy.

Conclusion

Delaying critical IIS updates significantly increases the possibility of unauthorized access and remote code execution. The organization should prioritize timely patching using controlled maintenance procedures. If immediate patching is

impossible, temporary network isolation and additional protective controls should be applied. High availability, backups, staging environments, phased deployment, and monitoring can minimize downtime while maintaining security.

4. Mobile Application – Plaintext Storage and Unsecured Communication

Problem Analysis

The mobile application stores sensitive information such as credentials and personal data in **plaintext** on the device. It also communicates with backend servers through unsecured channels.

These weaknesses expose both stored data and data transmitted across the network.

Exploitation Through Device Compromise

If a device is compromised, an attacker may potentially access application storage and retrieve plaintext information.

This can expose:

- Login credentials
- Personal information
- Authentication information
- Application data
- User account information

Plaintext storage therefore increases the impact of device theft, malware, or unauthorized device access.

Exploitation Through Network Interception

Unsecured communication can expose information while it travels between the mobile application and backend server.

An attacker positioned on an untrusted network could potentially intercept or manipulate communication.

This may result in:

- Credential exposure
- Personal-data leakage

- Session compromise
- Unauthorized requests
- Manipulation of application traffic

Impact on Privacy

The exposure of personal information can lead to:

- Identity-related fraud
- Account compromise
- Privacy violations
- Financial losses
- Customer complaints
- Regulatory problems

Impact on Organizational Reputation

A security breach can reduce customer confidence.

The organization may experience:

- Loss of customer trust
- Negative publicity
- Financial costs
- Incident-response expenses
- Legal or regulatory consequences
- Loss of business

Need for Encryption

Encryption protects sensitive information from unauthorized parties.

Two important areas must be protected:

Data at rest: Information stored on the device.

Data in transit: Information transmitted between the application and server.

Secure Storage Techniques

The application should:

- Avoid storing passwords in plaintext.
- Use secure OS-provided storage mechanisms.

- Store only information that is actually required.
- Protect sensitive tokens appropriately.
- Use strong authentication mechanisms.
- Apply secure key-management practices.

Secure Communication Techniques

The application should:

- Use **HTTPS/TLS** for all sensitive communication.
- Validate server certificates correctly.
- Avoid transmitting credentials through insecure protocols.
- Protect authentication tokens.
- Implement appropriate API authentication and authorization.
- Monitor suspicious communication patterns.

Conclusion

Plaintext storage and unsecured communication create serious security and privacy risks for a mobile application. An attacker who compromises the device could access locally stored information, while an attacker monitoring an insecure network could potentially intercept transmitted data. Therefore, sensitive data should be protected using secure device storage and strong encryption, while all communication with backend services should use HTTPS/TLS. These measures are essential for protecting users and maintaining organizational trust.

5. Windows Firewall Disabled

Problem Analysis

The Windows firewall was disabled temporarily for troubleshooting but was not re-enabled. Shortly afterward, unauthorized access attempts were detected.

The firewall normally provides an important layer of protection by controlling network connections to the system.

How Attackers Could Exploit the Exposure

When the firewall is disabled, services that would normally be protected may become reachable.

At a high level, an attacker could:

1. Discover the exposed Windows system.
2. Identify reachable network services.
3. Determine whether any exposed services contain vulnerabilities or weak configurations.
4. Attempt unauthorized authentication or exploitation.
5. If successful, obtain access to the system.
6. Attempt privilege escalation or access additional resources.

The risk depends on which services are exposed and whether they are properly secured.

Risks of Mismanaging Security Controls

Forgetting to restore the firewall can result in:

- Increased attack surface
- Unauthorized access attempts
- Exploitation of exposed services
- Malware infection
- Data compromise
- Privilege escalation
- Lateral movement
- Business disruption

The incident also demonstrates that **human error can weaken otherwise effective security controls**.

Importance of Automated Policies

Automated security policies reduce dependence on administrators remembering every manual change.

For example:

Firewall disabled → Temporary exception → Automatic expiration → Firewall restored

Security-management systems can also monitor firewall status and generate alerts whenever a critical protection mechanism is disabled.

Administrative Safeguards

The organization should implement:

1. **Change-management procedures** for firewall modifications.
2. **Time-limited firewall exceptions.**
3. Automatic restoration after troubleshooting.
4. Alerts when the firewall is disabled.
5. Centralized security-policy management.
6. Regular configuration audits.
7. Principle of least privilege for administrators.
8. Documented troubleshooting procedures.
9. Security monitoring and logging.
10. Regular administrator security training.

Defense-in-Depth Approach

The firewall should not be the only security control.

The organization should also use:

- Endpoint protection/EDR
- IDS/IPS
- Network segmentation
- Strong authentication
- Regular patching
- Secure service configurations
- Centralized logging
- Continuous monitoring

Conclusion

Disabling the Windows firewall temporarily can expose services that are normally protected and provide attackers with additional opportunities to gain unauthorized access. The incident highlights the risk of human error in security administration. Automated policies, time-limited exceptions, centralized monitoring, configuration auditing, and change-management procedures should be implemented. A defense-in-depth strategy ensures that even if one security control is temporarily disabled, other controls continue protecting critical systems.